

Robótica e Automação - Trabalho Nº3

Leonardo Soares da Costa Tanaka - DRE: 121067652

Engenharia de Controle e Automação/UFRJ

Rio de Janeiro, Brasil

Dezembro de 2024

1 Introdução

Para a resolução do trabalho, foi usado o manipulador Kinova Gen3 Ultra lightweight (www.kinovarobotics.com) da Kinova Inc.. A estrutura cinemática deste manipulador de 7 juntas pode ser vista na figura a seguir:

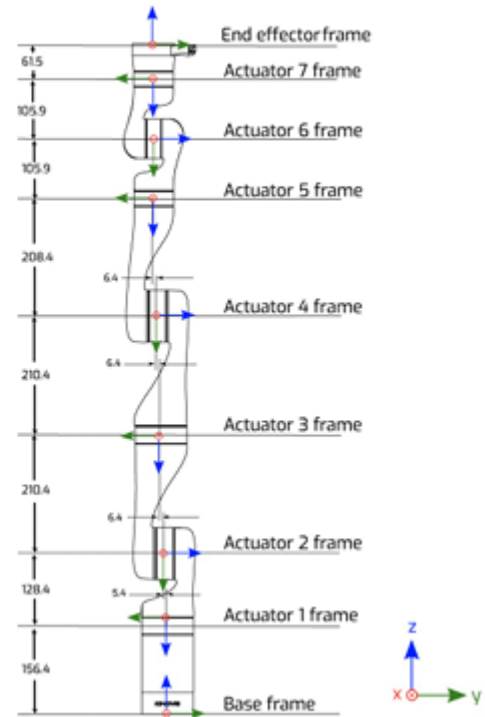


Figura 1: Foto e Diagrama do manipulador Kinova

Além disso, os parâmetros de Denavit-Hartenberg do manipulador são apresentados na Tabela 1. Foi considerado que a orientação do sistema de coordenadas 0 com relação à base é dado por $R_{b0} = R_x(\pi)$ onde $R_x(\cdot)$ é uma rotação elementar ao redor do eixo x.

Junta	α (rad)	A (m)	θ (rad)	D (m)	Offset (rad)
1	$\pi/2$	0	θ_1	-0.2848	π
2	$\pi/2$	0	θ_2	-0.0118	π
3	$\pi/2$	0	θ_3	-0.4208	π
4	$\pi/2$	0	θ_4	-0.0128	π
5	$\pi/2$	0	θ_5	-0.3143	π
6	$\pi/2$	0	θ_6	0	π
7	π	0	θ_7	0	π

Tabela 1: Parâmetros de Denavit-Hartenberg Standard do Kinova Gen3

Ademais, os parâmetros dinâmicos deste sistema robótico são dados no arquivo urdf, kinova_gen3.urdf, onde podem ser obtidos os parâmetros da massa, centro de massa e momento de inércia de cada elo no formato urdf. Podendo ser estruturados nas Tabelas 2 e 3:

Joint i	Parent	Child	xyz	rpy	Type	Axis
1	base_link	shoulder_link	0, 0, 0.15643	3.1416, 0, 0	continuous	0, 0, 1
2	shoulder_link	half_arm_1_link	0, 0.005375, -0.12838	1.5708, 0, 0	revolute	0, 0, 1
3	half_arm_1_link	half_arm_2_link	0, -0.21038, -0.006375	-1.5708, 0, 0	continuous	0, 0, 1
4	half_arm_2_link	forearm_link	0, 0.006375, -0.21038	1.5708, 0, 0	revolute	0, 0, 1
5	forearm_link	spherical_wrist_1_link	0, -0.20843, -0.006375	-1.5708, 0, 0	continuous	0, 0, 1
6	spherical_wrist_1_link	spherical_wrist_2_link	0, 0.00017505, -0.10593	1.5708, 0, 0	revolute	0, 0, 1
7	spherical_wrist_2_link	bracelet_link	0, -0.10593, -0.00017505	-1.5708, 0, 0	continuous	0, 0, 1
8	bracelet_link	end_effector_link	0, 0, -0.061525	3.14159, 0, 0	fixed	0, 0, 0

Tabela 2: URDF das juntas do manipulador Kinova

Link	Origem (xyz)	Massa(kg)	Inércia (ixx iyy izz ixz iyz)
base_link	-0.000648 -0.000166 0.084487	1.697	0.004622 0.004495 0.002079 9E-06 6E-05 9E-06
shoulder_link	-2.3E-05 -0.010364 -0.07336	1.377	0.00457 0.004831 0.001409 1E-06 2E-06 0.000448
half_arm_1_link	-4.4E-05 -0.09958 -0.013278	1.163	0.011088 0.001072 0.011255 5E-06 0 -0.000691
half_arm_2_link	-4.4E-05 -0.006641 -0.117892	1.163	0.010932 0.011127 0.001043 0 -7E-06 0.000606
forearm_link	-1.8E-05 -0.075478 -0.015006	0.93	0.008147 0.000631 0.008316 -1E-06 0 -0.0005
spherical_wrist_1_link	1E-06 -0.009432 -0.063883	0.678	0.001596 0.001607 0.000399 0 0 0.000256
spherical_wrist_2_link	1E-06 -0.045483 -0.00965	0.678	0.001641 0.00041 0.001641 0 0 -0.000278
bracelet_link	-9.3E-05 0.000132 -0.022905	0.364	0.000214 0.000223 0.00024 0 1E-06 -2E-06
end_effector_link	0 0 0	0	0 0 0 0 0 0

Tabela 3: URDF dos elos do manipulador Kinova com rpy="0 0 0" para todos os elos

Para saber a representação dos centros de massa nos sistemas de coordenadas do Denavit-Hartenberg que será usado para gerar o SerialLink, é necessário obter as transformações homogêneas entre os sistemas de coordenadas das duas representações a partir do Denavit-Hartenberg e do URDF. A seguir temos as transformações homogêneas:

$$T_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0.12837 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad T_2 = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0.006425 \\ 0 & 1 & 0 & -0.00001 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad T_3 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0.21041 \\ 0 & -1 & 0 & 0.00005 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (1)$$

$$T_4 = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0.006475 \\ 0 & 1 & 0 & 0.00003 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad T_5 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0.1059 \\ 0 & -1 & 0 & 0.0001 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad T_6 = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & -0.00007505 \\ 0 & 1 & 0 & -0.00003 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (2)$$

$$T_7 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & -0.0002501 \\ 0 & 0 & -1 & 0.10596 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (3)$$

Os parâmetros dinâmicos do robô incluem massa ($L(i).m$), centro de massa ($L(i).r$) e momentos de inércia ($L(i).I$) para cada elo. A massa de cada elo é especificada diretamente, enquanto o centro de massa é calculado utilizando as transformações homogêneas T_i aplicadas às coordenadas locais do elo, resultando em $\text{cmass}_i = T_i \cdot [x \ y \ z \ 1]^T$, com $L(i).r$ sendo os três primeiros elementos. Os momentos de inércia são definidos como um vetor contendo os valores em relação aos eixos principais e os produtos de inércia, usados na modelagem dinâmica. Além disso, a inércia dos motores é inicializada como $L(i).Jm = 0$ para todos os elos.

Para este manipulador, foi projetado os seguintes controladores:

1. Controlador PD:

$$\tau = K_p(\theta_d - \theta) + K_d(\dot{\theta}_d - \dot{\theta}), \quad (4)$$

de forma que os polos dominantes de cada elo tenham frequência natural $\omega_n = 10 \text{ rad/s}$ e coeficiente de amortecimento $\zeta = 1$.

2. Controlador P-PI:

$$\tau = K_d e_2 + K_i \int e_2 dt + K_a \ddot{\theta}_d, \quad (5)$$

onde

$$e_2 = K_p(\theta_d - \theta) + (\dot{\theta}_d - \dot{\theta}), \quad (6)$$

também garantindo $\omega_n = 10 \text{ rad/s}$ e $\zeta = 1$.

3. Controlador por Torque Computado + PD:

$$\tau = M(\theta)u + C(\theta, \dot{\theta})\dot{\theta} + G(\theta), \quad (7)$$

com

$$u = K_p(\theta_d - \theta) + K_d(\dot{\theta}_d - \dot{\theta}) + \ddot{\theta}_d, \quad (8)$$

assegurando $\omega_n = 10 \text{ rad/s}$ e $\zeta = 1$.

4. Controlador de Slotine & Li:

$$\tau = M(\theta)\ddot{\theta}_r + C(\theta, \dot{\theta})\dot{\theta}_r + G(\theta) - K_d\sigma, \quad (9)$$

onde

$$\sigma = (\dot{\theta} - \dot{\theta}_d) + \Lambda(\theta - \theta_d) = \dot{\theta} - \dot{\theta}_r, \quad (10)$$

garantindo desempenho similar aos itens anteriores.

A trajetória desejada, em radianos, é dada por:

$$\theta_{1d} = 0.5236 \cos(\omega_n t), \quad (11)$$

$$\theta_{2d} = 0.2618 + 0.2618 \cos(\omega_n t), \quad (12)$$

$$\theta_{3d} = \pi, \quad (13)$$

$$\theta_{4d} = -2.2689 + 0.2618 \cos(1.2\omega_n t), \quad (14)$$

$$\theta_{5d} = 0.0, \quad (15)$$

$$\theta_{6d} = 0.9599 + 0.2618 \cos(1.3\omega_n t), \quad (16)$$

$$\theta_{7d} = \pi/2. \quad (17)$$

O comportamento do sistema em malha fechada foi simulado para as condições iniciais do manipulador definidas pelos ângulos de juntas $\theta = [0, 15, 180, -130, 0, 55, 90]$ (graus), considerando as seguintes trajetórias desejadas:

- Trajetória rápida com $\omega_n = \pi$;
- Trajetória lenta com $\omega_n = \pi/2$.

Adicionalmente, foi analisado o caso em que a massa do último elo (*end-effector link*) muda para $m_l = 10$ kg. As simulações irão avaliar a estabilidade e o desempenho de cada controlador projetado sob essa nova condição. Isso foi seguido conforme a mudança da massa, o deslocamento do centro de massa e a mudança da inércia do último elo que foram implementados seguindo as seguintes formulações:

$$m_7 = 10 + 0.364 = 10.364, \quad (18)$$

$$r = \frac{0.364 \cdot \begin{bmatrix} -0.0001 \\ -0.0004 \\ 0.1289 \end{bmatrix} + 10 \cdot \begin{bmatrix} 0 \\ 0 \\ 0.1674 \end{bmatrix}}{10.364} \quad (19)$$

$$I_P = I_c - m (\vec{r}_c \vec{r}_c \cdot - ||\vec{r}_c||^2 I) \quad (20)$$

2 Desenvolvimento

Esta seção propõe o projeto e análise de quatro estratégias de controle – PD, P-PI, Torque Computado com PD e Slotine & Li – ajustadas para assegurar pólos dominantes com frequência natural $\omega_n = 10$ rad/s e coeficiente de amortecimento $\zeta = 1$. As simulações consideram trajetórias rápidas ($\omega_n = \pi$) e lentas ($\omega_n = \pi/2$), além de mudanças no manipulador, como o aumento da massa do último elo para $m_{l2} = 10$ kg. O objetivo é avaliar o desempenho e a estabilidade de cada controlador, destacando suas vantagens e limitações em diferentes cenários operacionais. O polinômio característico associado aos pólos dominantes desejados é dado por:

$$p_c(s) = s^2 + 20s + 100 \quad (21)$$

2.1 Controlador PD

Considera-se que $M(\theta)$ é expressa como:

$$M(\theta) = \bar{M} + \Delta M(\theta), \quad (22)$$

em que $\bar{M} \in \mathbb{R}^{n \times n}$ é uma matriz positiva, diagonal e constante, i.e.,

$$\bar{M} = \text{diag}(\bar{m}_1, \dots, \bar{m}_n) > 0, \quad \bar{m}_i \in \mathbb{R}. \quad (23)$$

Para seu cálculo, cada elemento diagonal $\bar{m}_i$ é determinado pela soma dos momentos de inércia ($L(j).I(3,3)$) e das contribuições das massas dos elos ($L(j).m$), ponderadas pela distância do centro de massa ($\|\mathbf{r}_j\|^2$) e a distância acumulada das juntas anteriores até o elo atual. A distância acumulada é calculada como o quadrado do deslocamento das juntas ($|L(j).d|^2$). Dessa forma, considerando o modelo dinâmico do manipulador, tem-se:

$$\tau = (\bar{M} + \Delta M(\theta))\ddot{\theta} + C(\theta, \dot{\theta})\dot{\theta} + G(\theta) \quad (24)$$

Portanto, o modelo dinâmico do manipulador pode ser escrito como:

$$\tau = \bar{M}\ddot{\theta} + d, \quad (25)$$

em que: $d = \Delta M(\theta)\ddot{\theta} + C(\theta, \dot{\theta})\dot{\theta} + G(\theta)$.

Ao utilizar o controlador PD ($u = -K_d\dot{\theta} - K_p\tilde{\theta}$) é obtido o seguinte polinômio característico do manipulador:

$$p_c(s) = \bar{M}s^2 + K_d s + K_p \quad (26)$$

em que $K_d = 20\bar{M}$ e $K_p = 100\bar{M}$ para respeitar as especificações do projeto. A seguir temos o diagrama do controlador PD com os ganhos projetados implementado no Simulink:

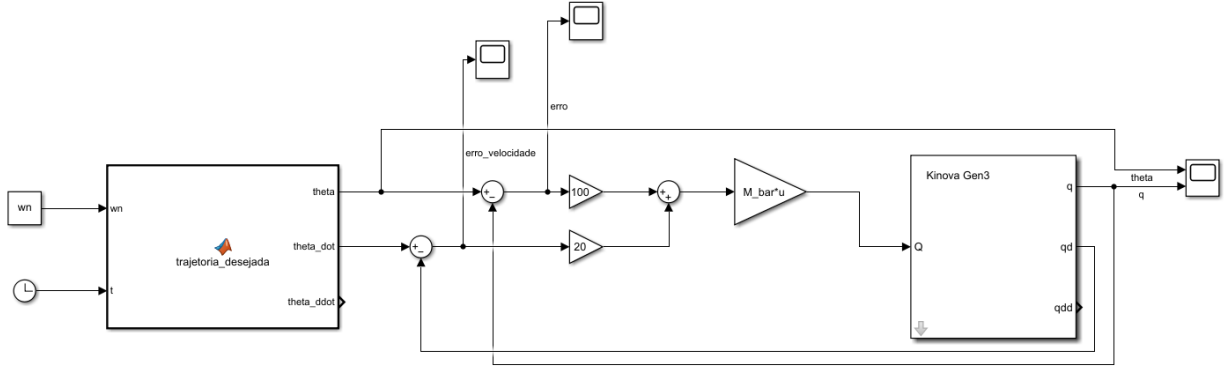
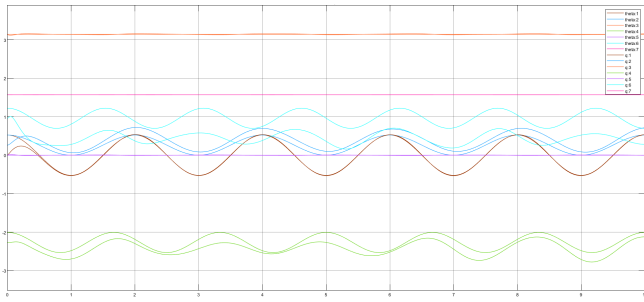
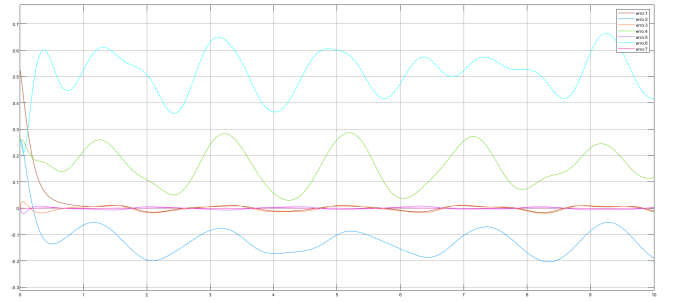


Figura 2: Diagrama do controlador PD

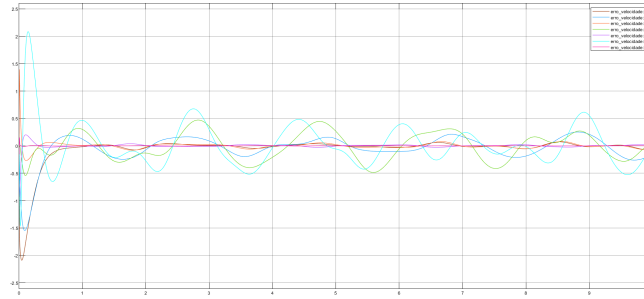
Além disso, foi avaliado esse controlador para um intervalo de 10 segundos com um $\omega_n = \pi$. Isso foi feito observando as posições das juntas, o erro entre as posições das juntas e as posições das trajetórias desejadas e o erro entre as velocidades das juntas e as velocidades das trajetórias desejadas. A seguir temos os gráficos:



(a) Posições das juntas



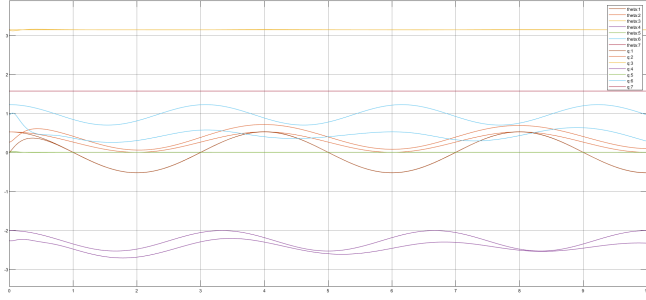
(b) Erros de posições das juntas



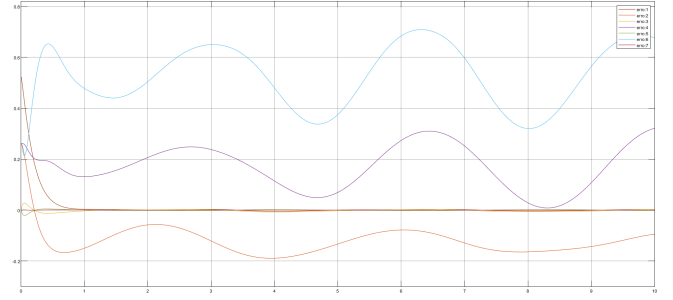
(c) Erros de velocidades das juntas

Figura 3: Controlador PD ($\omega_n = \pi$)

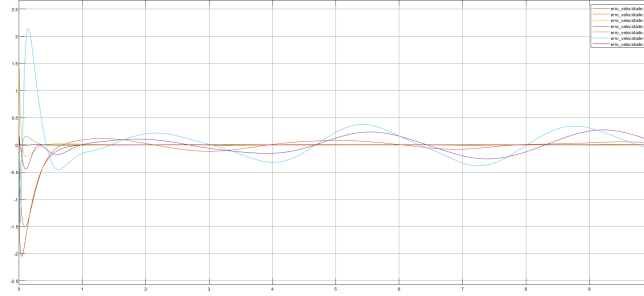
Isso também foi feito para $\omega_n = \pi/2$. Obtendo os seguintes resultados:



(a) Posições das juntas



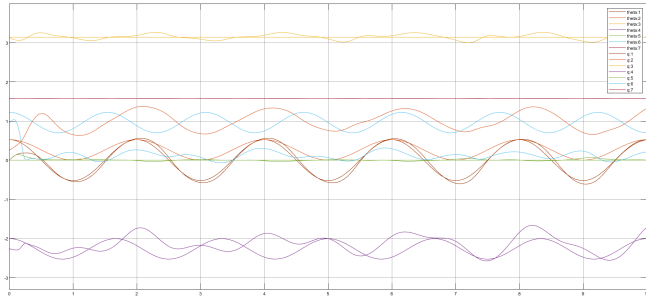
(b) Erros de posições das juntas



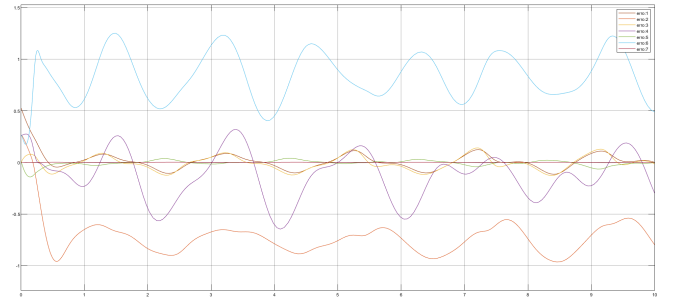
(c) Erros de velocidades das juntas

Figura 4: Controlador PD ($\omega_n = \pi/2$)

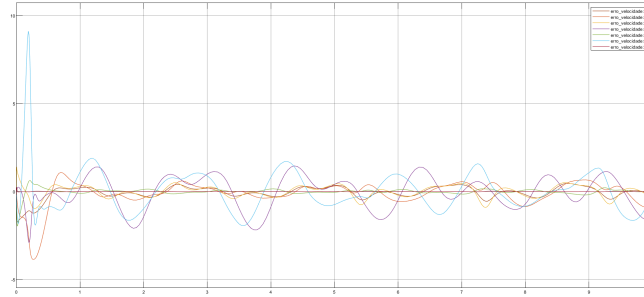
Além disso, foi feita a simulação com a massa adicional de 10kg para $\omega_n = \pi$:



(a) Posições das juntas



(b) Erros de posições das juntas



(c) Erros de velocidades das juntas

Figura 5: Controlador PD com massa adicional ($\omega_n = \pi$)

Ademais, foi feita a simulação com a massa adicional de 10kg para $\omega_n = \pi/2$:

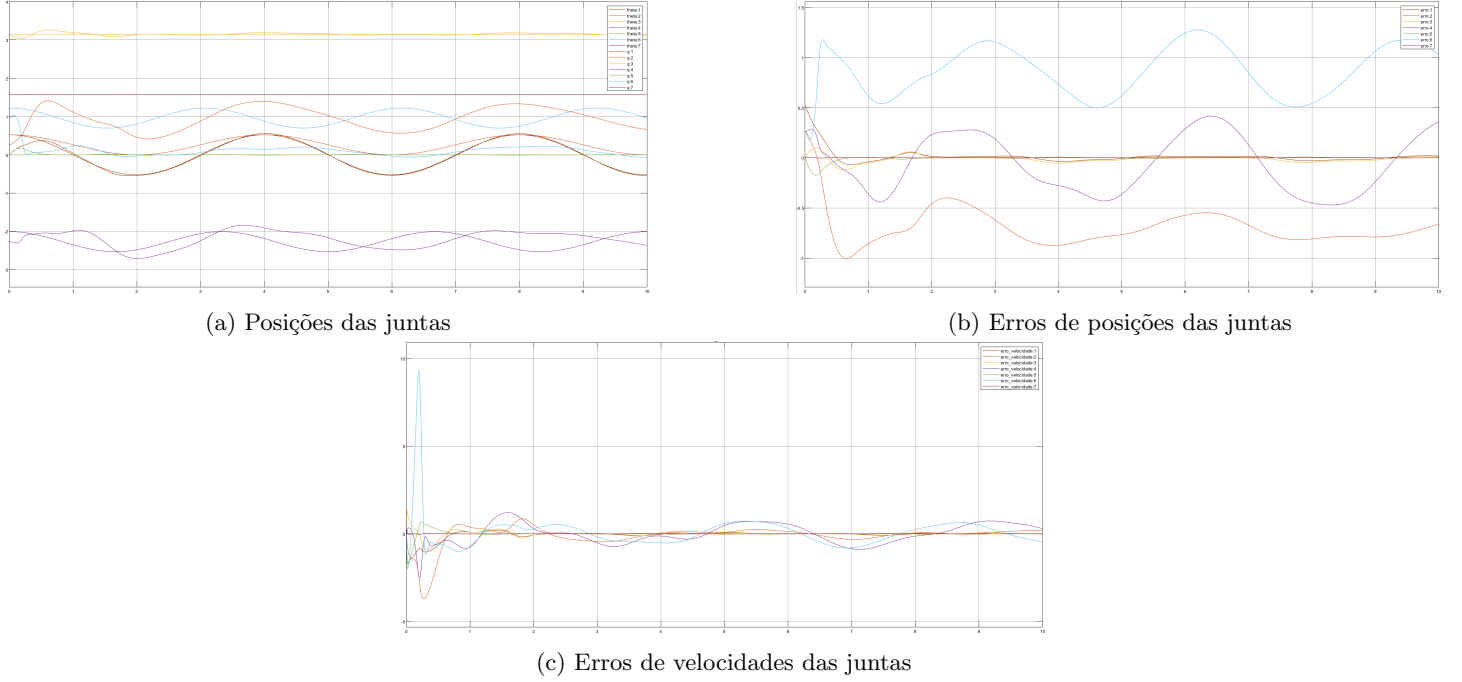


Figura 6: Controlador PD com massa adicional ($\omega_n = \pi/2$)

2.2 Controlador P-PI

Utilizando o modelo dinâmico do manipulador da equação (25) com as equações do controlador P-PI (5) e (6) com $K_a = \bar{M}$, foi obtido o seguinte desenvolvimento:

$$\tau = \bar{M}\ddot{\theta} + d = -K_d K_p \tilde{\theta} - K_d \dot{\tilde{\theta}} - K_i K_p \int \tilde{\theta} dt - K_i \tilde{\theta} + \bar{M}\ddot{\theta}_d \quad (27)$$

em que d é considerado uma perturbação e $\tilde{\theta} = \theta - \theta_d$.

A equação característica do sistema em malha fechada foi obtido a partir da Transformação de Laplace da equação acima e é escrita como:

$$p_c(s) = \bar{M}s^3 + K_d s^2 + (K_d K_p + K_i)s + K_i K_p = 0 \quad (28)$$

A partir disso é desejado a seguinte equação característica respeitando os parâmetros de projeto:

$$p_c(s) = \bar{M}(s^2 + 100s + 2500)(s + 10) = \bar{M}(s^3 + 110s^2 + 3500s + 25000) \quad (29)$$

com isso temos que $K_d = 110\bar{M}$, $K_p = 10.8271\bar{M}$ e $K_i = 2309\bar{M}$.

A seguir temos o diagrama do controlador P-PI implementado no Simulink:

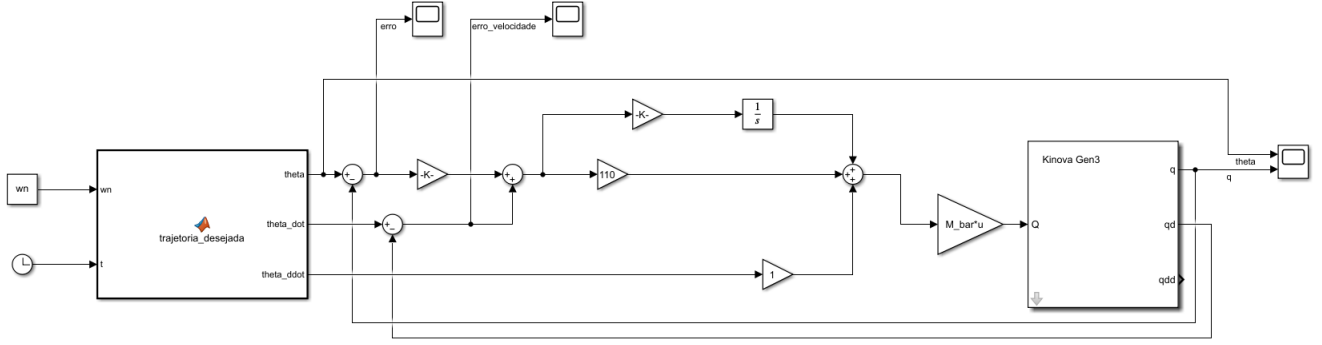


Figura 7: Diagrama do controlador P-PI

Além disso, foi avaliado esse controlador para um intervalo de 2π segundos com um $\omega_n = \pi$. Isso foi feito observando as posições das juntas, o erro entre as posições das juntas e as posições das trajetórias desejadas e o erro entre as velocidades das juntas e as velocidades das trajetórias desejadas. A seguir temos os gráficos:

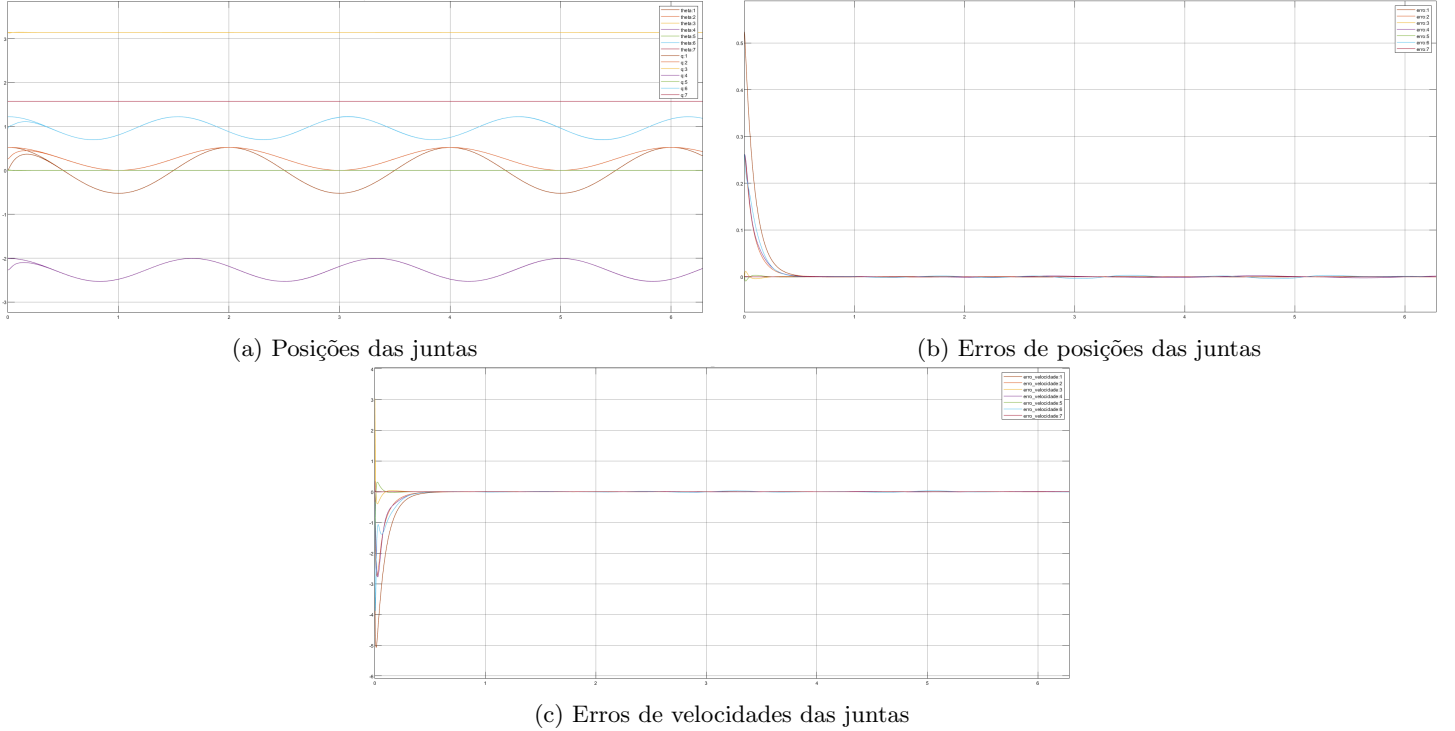


Figura 8: Controlador P-PI ($\omega_n = \pi$)

Isso também foi feito para $\omega_n = \pi/2$. Obtendo os seguintes resultados:

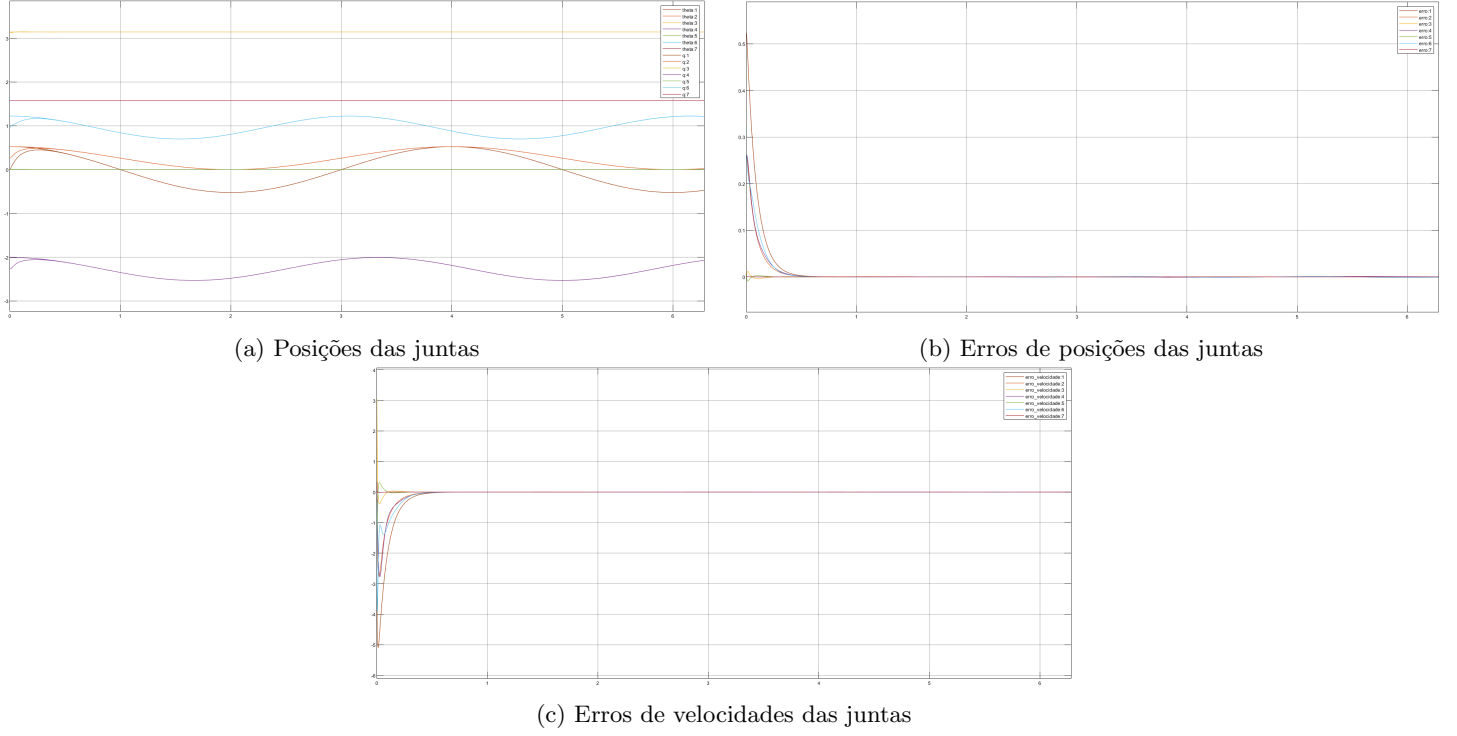


Figura 9: Controlador P-PI ($\omega_n = \pi/2$)

Além disso, foi feita a simulação com a massa adicional de 10kg para $\omega_n = \pi$:

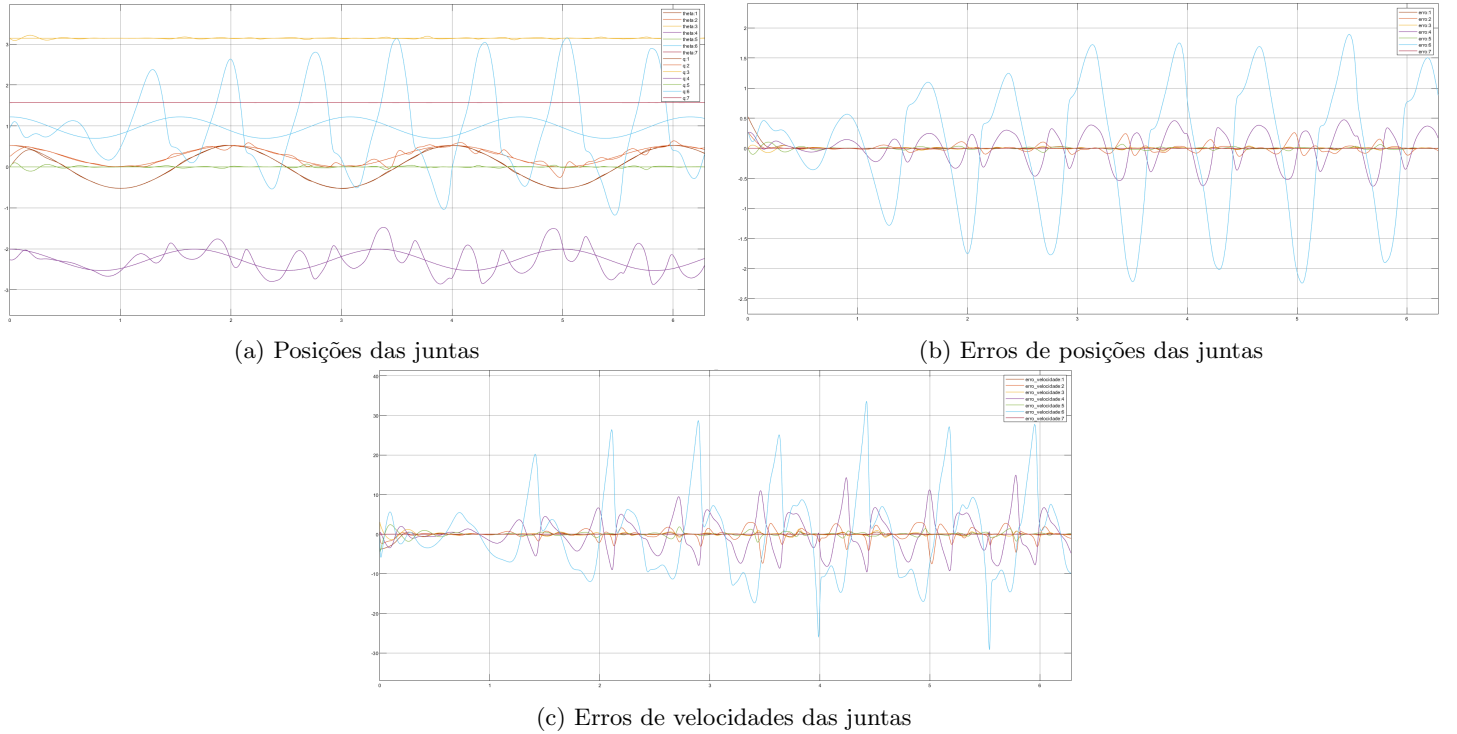


Figura 10: Controlador P-PI com massa adicional ($\omega_n = \pi$)

Ademais, foi feito a simulação com a massa adicional de 10kg para $\omega_n = \pi/2$:

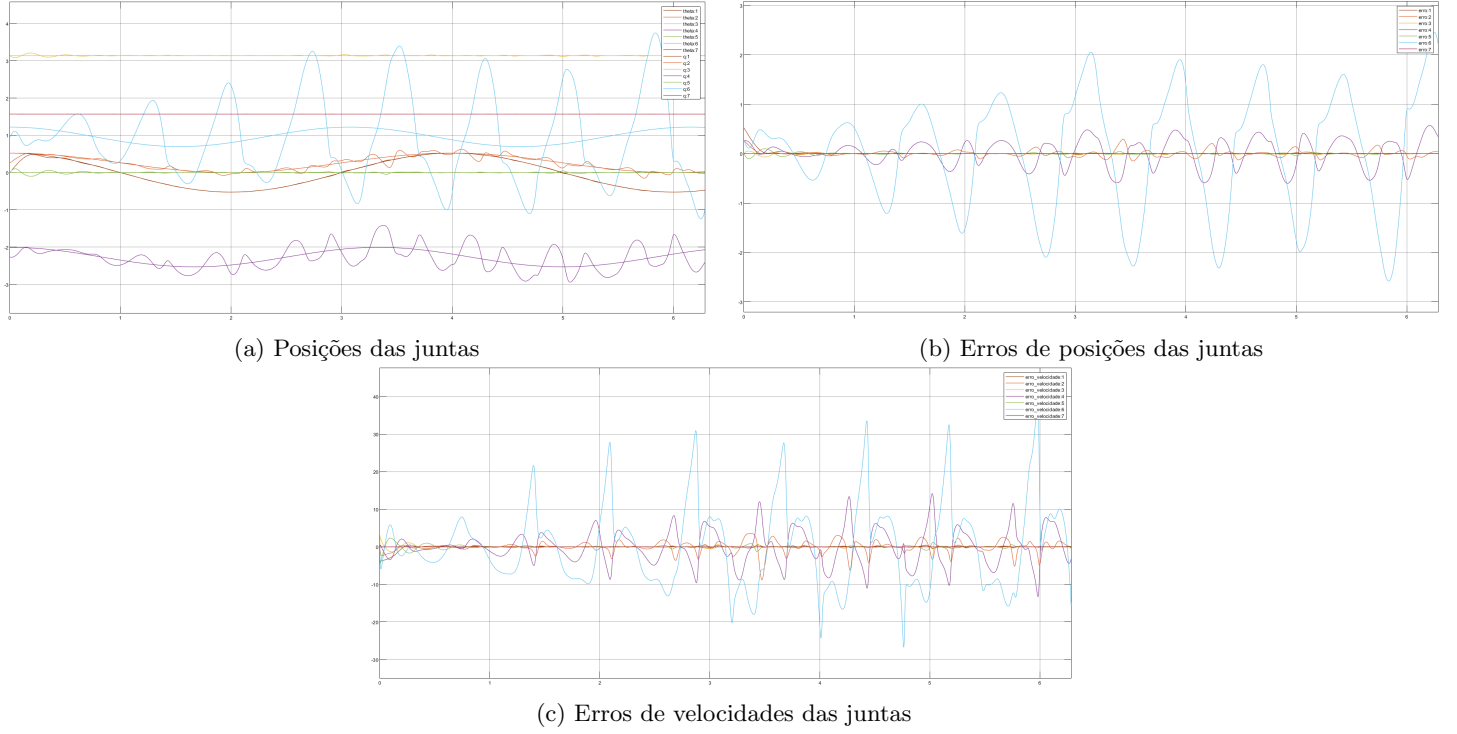


Figura 11: Controlador P-PI com massa adicional ($\omega_n = \pi/2$)

2.3 Controlador por Torque Computado + PD

É possível cancelar as não linearidades ao utilizar o torque computado (linearização por realimentação):

$$\tau = M(\theta)[\ddot{\theta}_d + u] + C(\theta, \dot{\theta})\dot{\theta} + G(\theta), \quad (30)$$

obtém-se o sistema em malha fechada com o erro definido como $\tilde{\theta} = \theta - \theta_d$:

$$\ddot{\tilde{\theta}} = u. \quad (31)$$

Ao se utilizar o controlador PD ($u = -K_p\tilde{\theta} - K_d\dot{\tilde{\theta}}$) no sistema acima, o erro do sistema é dado por:

$$\ddot{\tilde{\theta}} + K_d\dot{\tilde{\theta}} + K_p\tilde{\theta} = 0. \quad (32)$$

em que $K_d = 20$ e $K_p = 100$ para respeitar as especificações do projeto.

A seguir temos o diagrama do controlador por Torque Computado + PD implementado no Simulink:

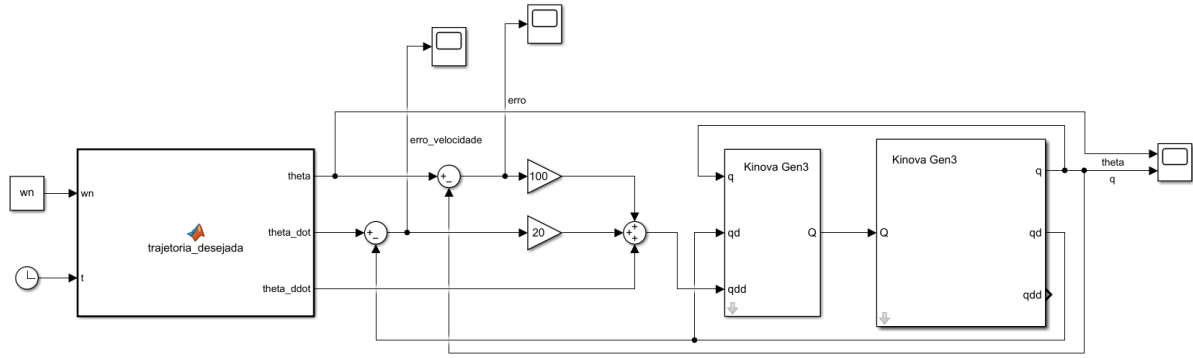
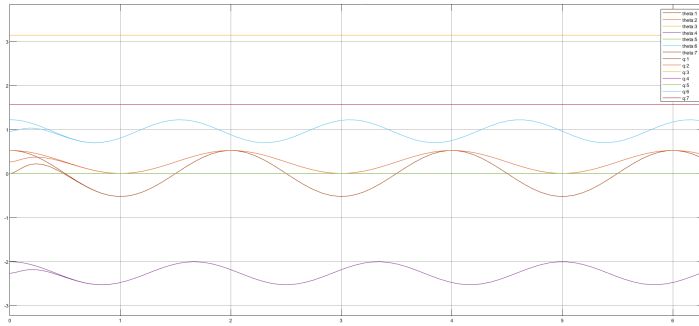


Figura 12: Diagrama do controlador por Torque Computado + PD

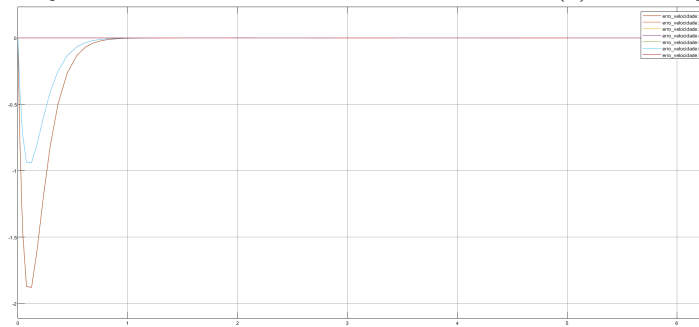
Além disso, foi avaliado esse controlador para um intervalo de 2π segundos com um $\omega_n = \pi$. Isso foi feito observando as posições das juntas, o erro entre as posições das juntas e as posições das trajetórias desejadas e o erro entre as velocidades das juntas e as velocidades das trajetórias desejadas. A seguir temos os gráficos:



(a) Posições das juntas



(b) Erros de posições das juntas



(c) Erros de velocidades das juntas

Figura 13: Controlador por Torque Computado + PD ($\omega_n = \pi$)

Isso também foi feito para $\omega_n = \pi/2$. Obtendo os seguintes resultados:

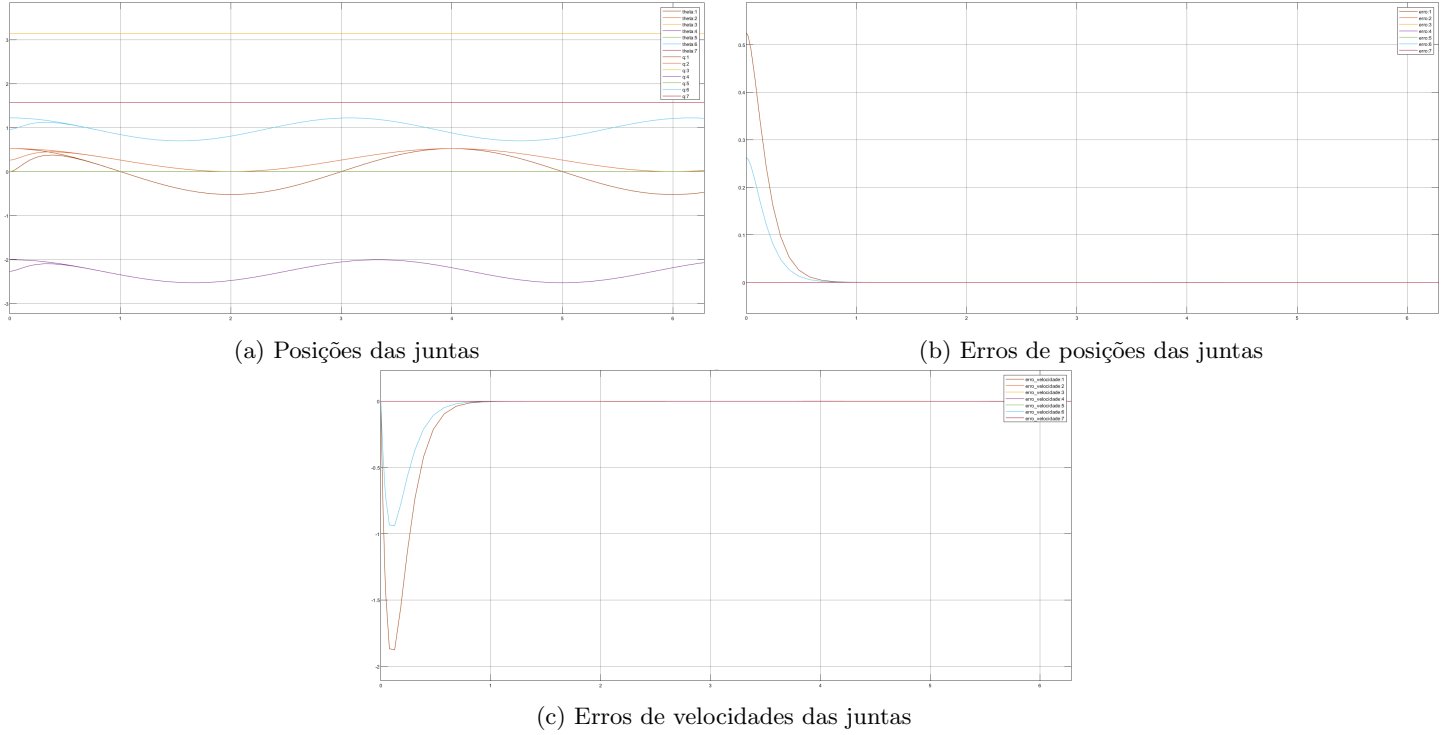


Figura 14: Controlador por Torque Computado + PD ($\omega_n = \pi/2$)

Além disso, foi feita a simulação com a massa adicional de 10kg para $\omega_n = \pi$:

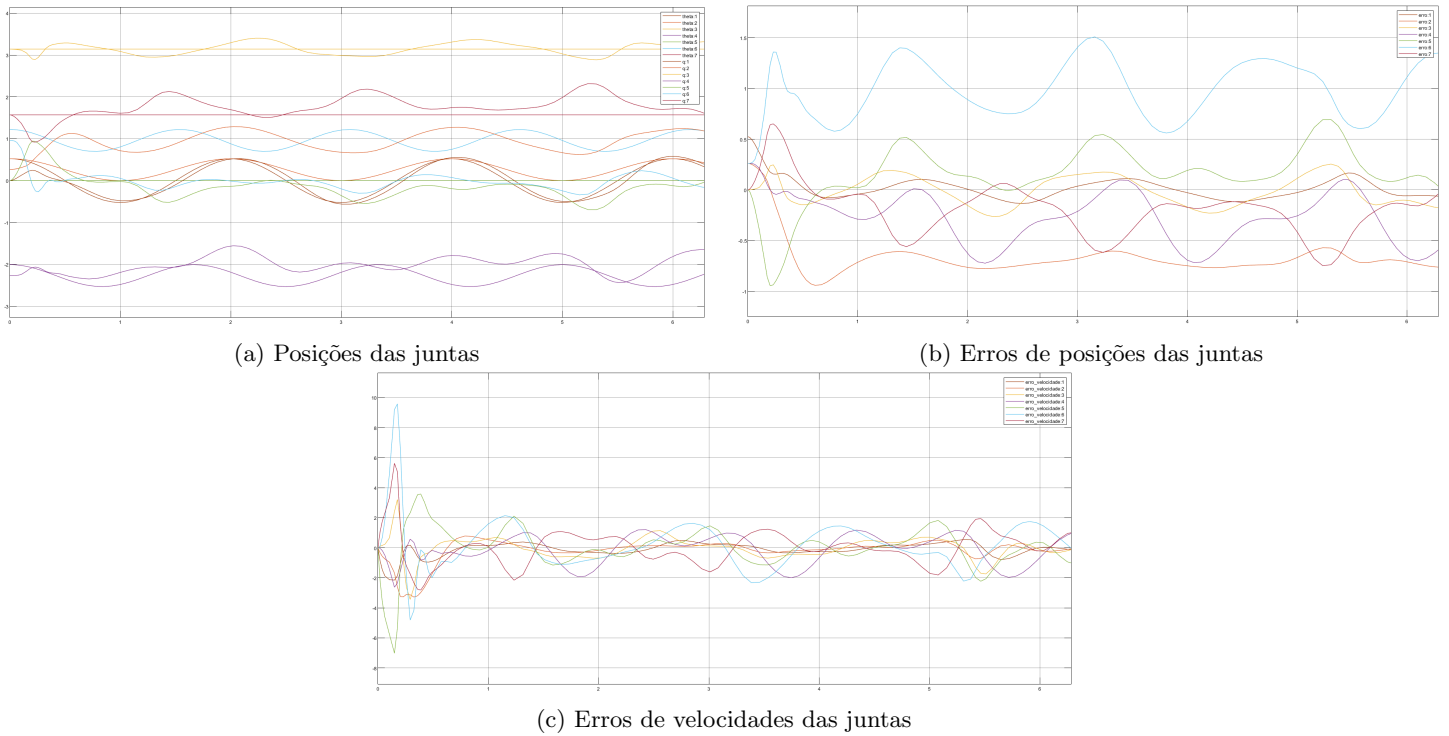


Figura 15: Controlador por Torque Computado + PD ($\omega_n = \pi$)

Ademais, foi feita a simulação com a massa adicional de 10kg para $\omega_n = \pi/2$:

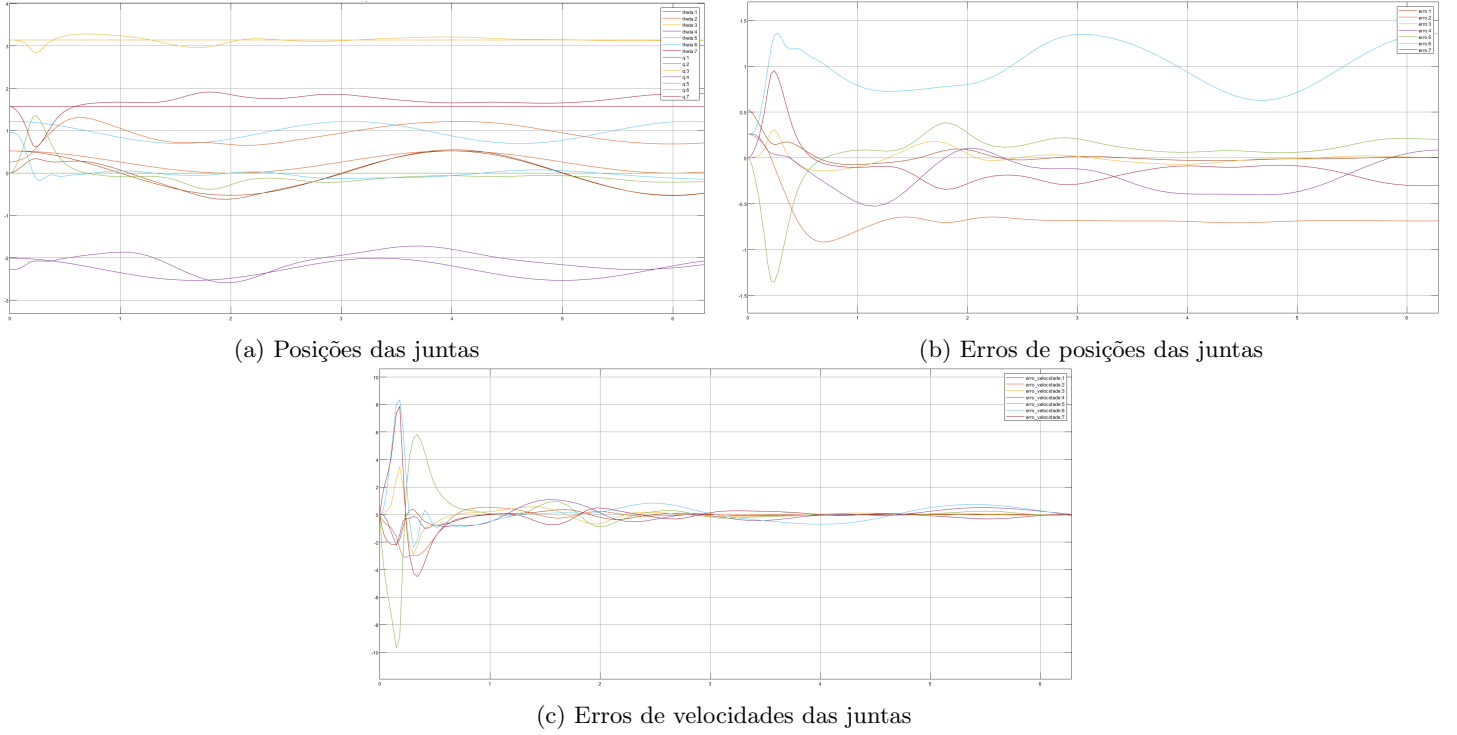


Figura 16: Controlador por Torque Computado + PD ($\omega_n = \pi/2$)

2.4 Controlador de Slotine & Li

Para validar o modelo de controle apresentado, foi realizado uma série de experimentos ajustando os parâmetros K_d e Λ com o objetivo de alcançar um desempenho próximo ao projetado. Após diversas iterações, os valores $K_d = 5$ e $\Lambda = 10$ foram escolhidos, pois proporcionaram uma resposta do sistema estável e adequada ao critério de desempenho definido. A lei de controle utilizada foi implementada conforme descrito anteriormente, com σ sendo ajustada para minimizar o erro de rastreamento e garantir o cumprimento das especificações. Os valores de K_d e Λ foram ajustados experimentalmente, buscando uma resposta dinâmica consistente com o comportamento projetado. Obtendo o seguinte diagrama:

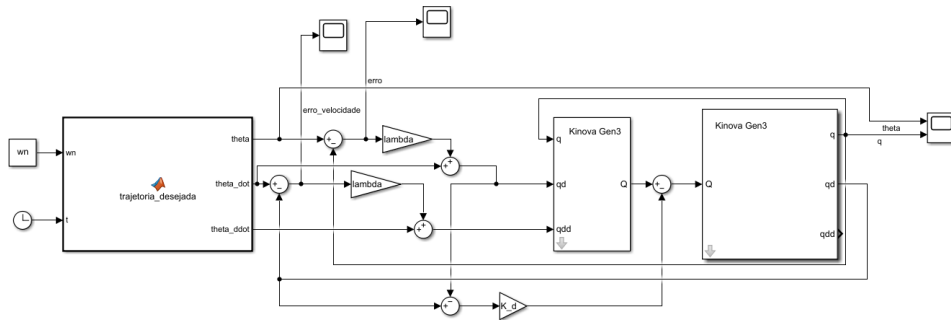


Figura 17: Diagrama do controlador de Slotine & Li

Então, foi feita a simulação para $\omega_n = \pi$. Obtendo os seguintes resultados:

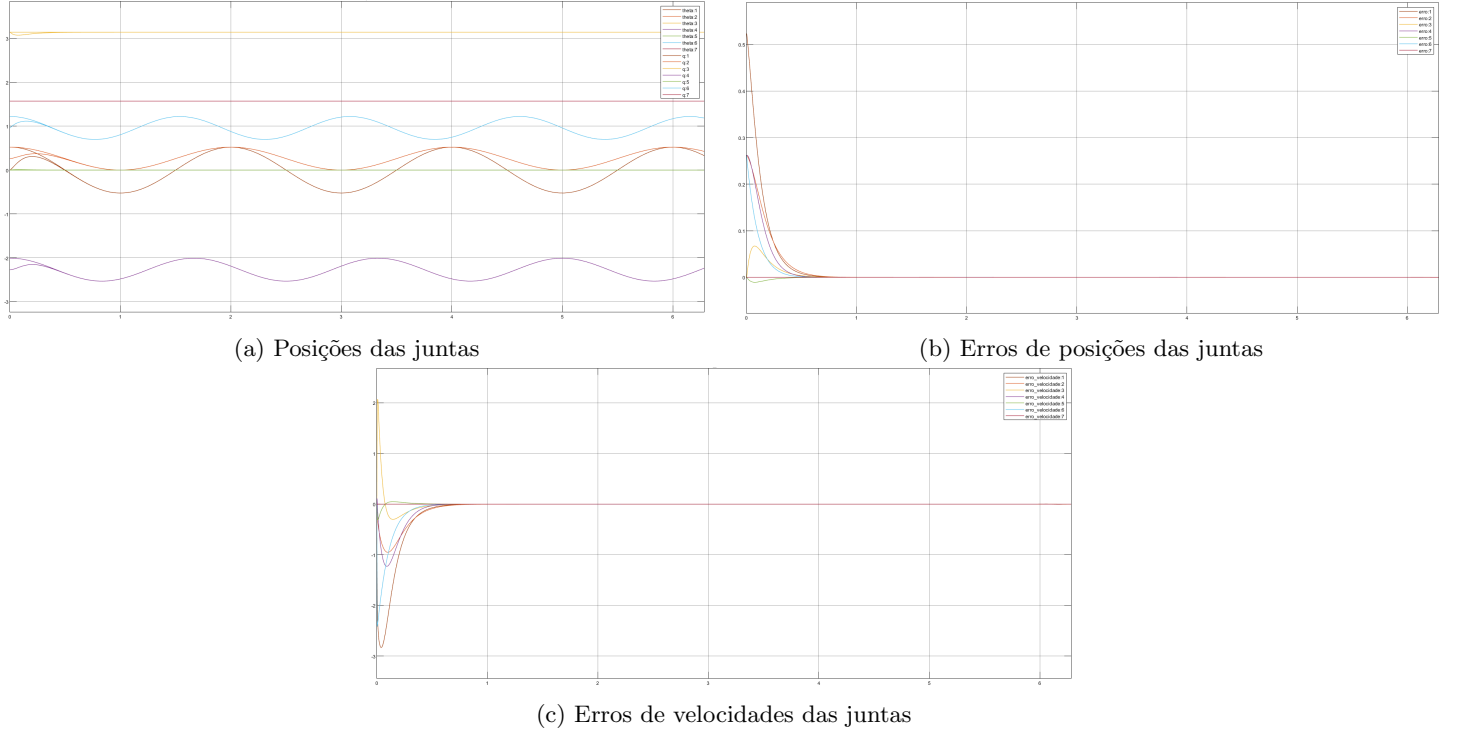


Figura 18: Controlador por Slotine & Li ($\omega_n = \pi$)

Isso também foi feito para $\omega_n = \pi/2$. Obtendo os seguintes resultados:

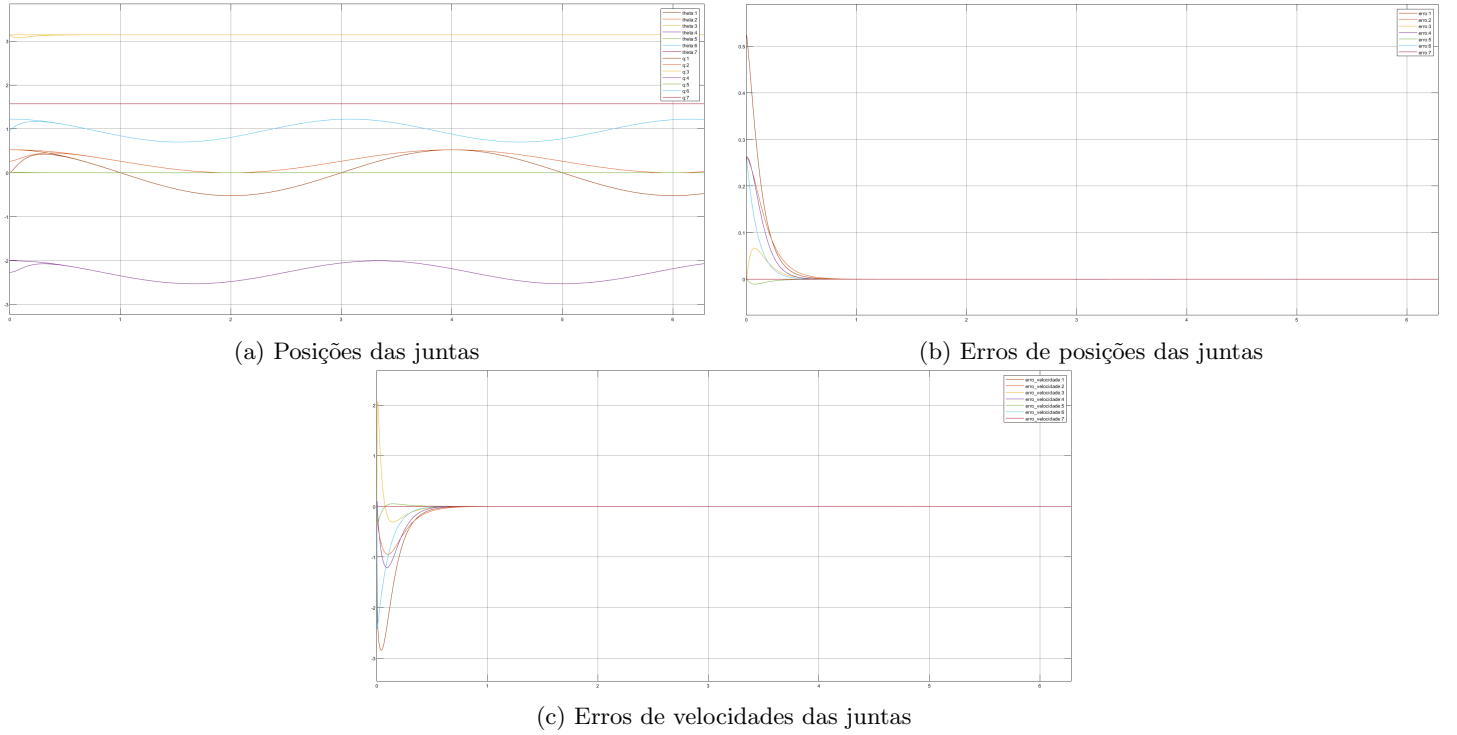


Figura 19: Controlador por Slotine & Li ($\omega_n = \pi/2$)

Além disso, foi feita a simulação com a massa adicional de 10kg para $\omega_n = \pi$:

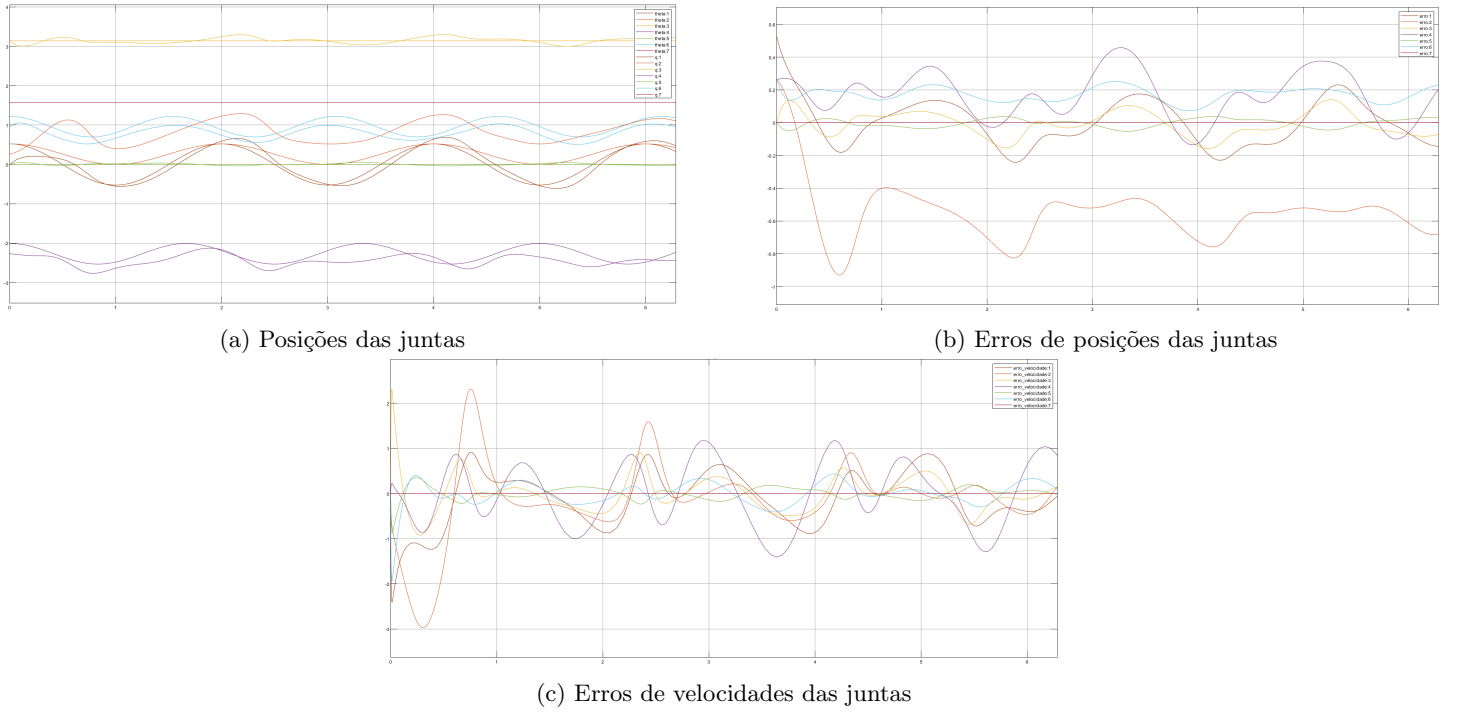


Figura 20: Controlador por Slotine & Li ($\omega_n = \pi$)

Ademais, foi feita a simulação com a massa adicional de 10kg para $\omega_n = \pi/2$:

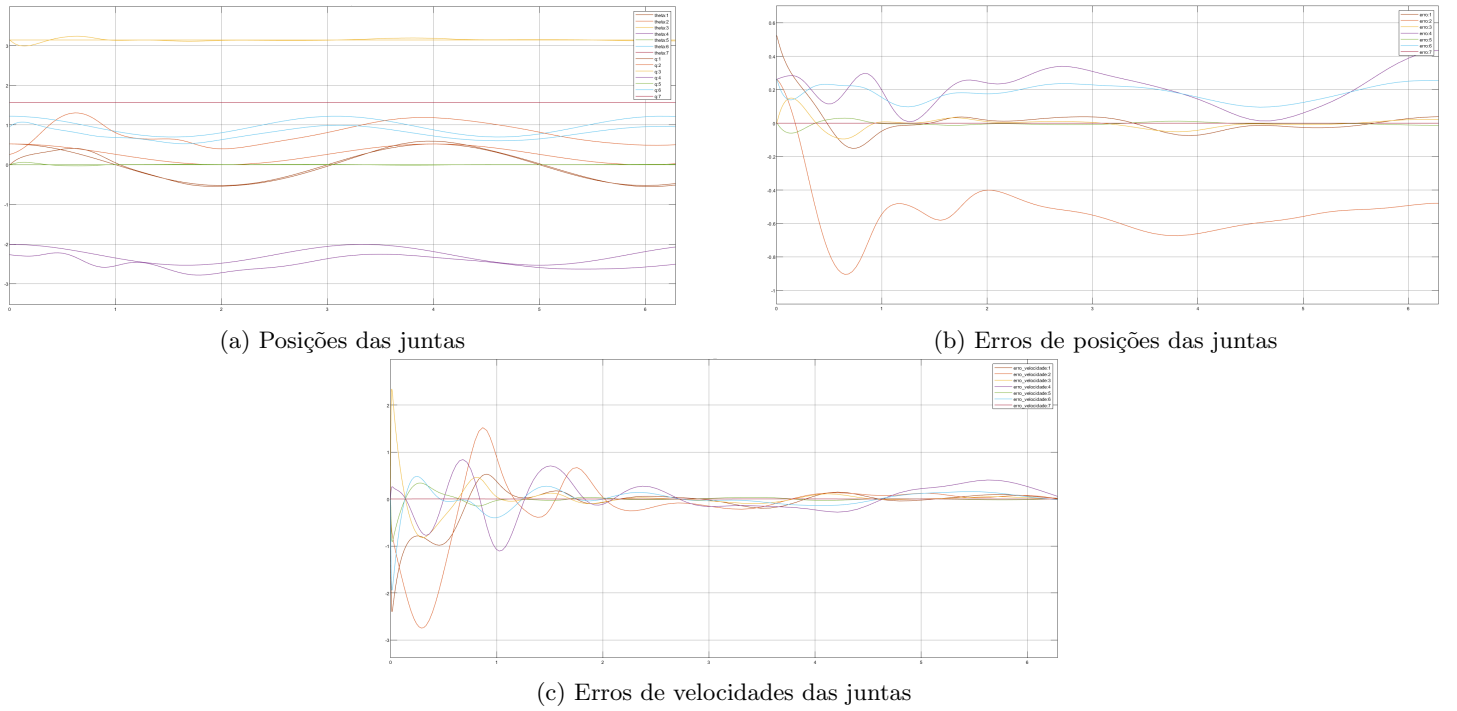


Figura 21: Controlador por Slotine & Li ($\omega_n = \pi/2$)

3 Conclusão

Após a implementação e simulação dos controladores PD, P-PI, Torque Computado + PD e Slotine & Li no manipulador Kinova Gen3, considerando trajetórias rápidas e lentas, e a alteração de massa do último elo, conclui-se que quase todos os controladores demonstraram desempenho estável e adequado em condições nominais, mas o controlador PD não teve um desempenho adequado por não conseguir seguir as trajetórias por não ter uma parcela integradora para zerar o erro em regime permanente e diferiram em robustez frente à alteração de massa.

Ao adicionar uma massa de 10 kg no último efetuador, os controladores PD e P-PI apresentaram uma piora considerável no quesito estabilidade. O Torque Computado + PD conseguiu manter a estabilidade, mas teve um desempenho inferior ao controlador de Slotine & Li, que também manteve a estabilidade, embora com um desempenho bem reduzido em relação ao projetado.

Os controladores P-PI e por Torque Computado + PD apresentaram melhor desempenho em rastreamento de trajetórias, devido, respectivamente, à parcela integradora do controlador PI e à compensação explícita de termos dinâmicos, enquanto o controlador de Slotine & Li destacou-se pela robustez em cenários de incerteza dinâmica. Por outro lado, o controlador PD apresentou oscilações e menor precisão nas condições modificadas.

Apêndice

Listing 1: Código principal do trabalho

```

1 % Definindo a cinematica direta do Kinova Gen3 usando o padrao DH (Denavit-Hartenberg)
2 clear L
3 % Definicao dos parametros DH para cada elo do robo
4 %           theta    d(m)    a    alpha type offset
5 L(1) = Link([ 0      -0.2848  0      pi/2  0    0], 'standard');
6 L(2) = Link([ 0      -0.0118  0      pi/2  0    pi], 'standard');
7 L(3) = Link([ 0      -0.4208  0      pi/2  0    pi], 'standard');
8 L(4) = Link([ 0      -0.0128  0      pi/2  0    pi], 'standard');
9 L(5) = Link([ 0      -0.3143  0      pi/2  0    pi], 'standard');
10 L(6) = Link([ 0      0        0      pi/2  0    pi], 'standard');
11 L(7) = Link([ 0      0        0      pi    0    pi], 'standard');
12
13 % Convertendo os frames do URDF para os frames do padrao DH
14 % Definicao das transformacoes homogeneas para alinhar os frames
15 % Frame 1
16 R1 = rotx(-90);
17 P1 = [0;0.12837;0];
18 T1 = SE3(R1, P1);
19
20 % Frame 2
21 R2 = rotx(-90)*rotz(180);
22 P2 = [0;0.006425; -0.00001];
23 T2 = SE3(R2, P2);
24
25 % Frame 3
26 R3 = rotx(-90);
27 P3 = [0;0.21041;0.00005];
28 T3 = SE3(R3, P3);
29
30 % Frame 4
31 R4 = roty(180)*rotx(-90);
32 P4 = [0;0.006475;0.00003];
33 T4 = SE3(R4, P4);
34
35 % Frame 5
36 R5 = rotx(-90);
37 P5 = [0;0.1059;0.0001];
38 T5 = SE3(R5, P5);
39
40 % Frame 6
41 R6 = roty(180)*rotx(-90);
42 P6 = [0;-0.00007505;-0.00003];

```

```

43 T6 = SE3(R6, P6);
44
45 % Frame 7
46 R7 = rotx(180);
47 P7 = [0; -0.0002501; 0.10596];
48 T7 = SE3(R7, P7);
49
50 % Definicao dos parametros dinamicos do robo
51 % Massa, centro de massa e inercia para cada elo
52 L(1).m = 1.377; % Massa do shoulder_link
53 cmass1 = T1.T*[-2.3E-05; -0.010364; -0.07336; 1];
54 L(1).r = cmass1(1:3); % Centro de massa
55 L(1).I = [0.00457, 0.004831, 0.001409, 1E-06, 0.000448, 2E-06]; % Inercia
56
57 L(2).m = 1.163; % Massa do half_arm_1_link
58 cmass2 = T2.T*[-4.4E-05; -0.09958; -0.013278; 1];
59 L(2).r = cmass2(1:3);
60 L(2).I = [0.011088, 0.001072, 0.011255, 5E-06, -0.000691, 0];
61
62 L(3).m = 1.163; % Massa do half_arm_2_link
63 cmass3 = T3.T*[-4.4E-05; -0.006641; -0.117892; 1];
64 L(3).r = cmass3(1:3);
65 L(3).I = [0.010932, 0.011127, 0.001043, 0, 0.000606, -7E-06];
66
67 L(4).m = 0.93; % Massa do forearm_link
68 cmass4 = T4.T*[-1.8E-05; -0.075478; -0.015006; 1];
69 L(4).r = cmass4(1:3);
70 L(4).I = [0.008147, 0.000631, 0.008316, -1E-06, -0.0005, 0];
71
72 L(5).m = 0.678; % Massa do spherical_wrist_1_link
73 cmass5 = T5.T*[1E-06; -0.009432; -0.063883; 1];
74 L(5).r = cmass5(1:3);
75 L(5).I = [0.001596, 0.001607, 0.000399, 0, 0.000256, 0];
76
77 L(6).m = 0.678; % Massa do spherical_wrist_2_link
78 cmass6 = T6.T*[1E-06; -0.045483; -0.00965; 1];
79 L(6).r = cmass6(1:3);
80 L(6).I = [0.001641, 0.00041, 0.001641, 0, -2E-06, 0];
81
82 L(7).m = 0.364; % Massa do bracelet_link + end-effector
83 cmass7 = T7.T*[-9.3E-05; 0.000132; -0.022905; 1];
84 L(7).r = cmass7(1:3);
85 L(7).I = [0.000214, 0.000223, 0.00024, 0, -2E-06, 1E-06];
86
87 % Inicializar a inercia do motor para todas as juntas como zero
88 for i = 1:7

```

```

89     L(i).Jm = 0;
90 end
91
92 % Criando o modelo dinamico do robo Kinova Gen3
93 kinova = SerialLink(L, 'name', 'Kinova Gen3');
94
95 % Definir a transformacao base do robo
96 Tb0 = trotx(180);
97 kinova.base = Tb0;
98
99 % Inicializar a matriz M_bar como uma matriz diagonal
100 M_bar = zeros(7, 7);
101
102 % Calcular a matriz de inercia diagonal M_bar
103 for i = 1:7
104     % Inicializar o termo diagonal da junta i
105     M_bar_i = 0;
106     % Acumulador para a distancia ao longo das juntas
107     distancia_acumulada = 0;
108     for j = i:7
109         % Adicionar o termo de inercia e centro de massa
110         M_bar_i = M_bar_i + L(j).I(3,3) + L(j).m * (norm(L(j).r)^2 + (distancia_acumulada));
111
112         % Distancia acumulada ate a junta atual
113         distancia_acumulada = distancia_acumulada + abs(L(j).d)^2; % Somar comprimento da junta
114     end
115     % Preencher o elemento diagonal correspondente
116     M_bar(i, i) = M_bar_i;
117 end
118
119 clear L_alterado
120
121 % Definicao dos parametros DH para cada elo do robo
122 %           theta    d(m)    a    alpha type offset
123 L_alterado(1) = Link([ 0    -0.2848  0    pi/2  0    0], 'standard');
124 L_alterado(2) = Link([ 0    -0.0118  0    pi/2  0    pi], 'standard');
125 L_alterado(3) = Link([ 0    -0.4208  0    pi/2  0    pi], 'standard');
126 L_alterado(4) = Link([ 0    -0.0128  0    pi/2  0    pi], 'standard');
127 L_alterado(5) = Link([ 0    -0.3143  0    pi/2  0    pi], 'standard');
128 L_alterado(6) = Link([ 0    0        0    pi/2  0    pi], 'standard');
129 L_alterado(7) = Link([ 0    0        0    pi    0    pi], 'standard');
130
131 % Massa, centro de massa e inercia para cada elo
132 L_alterado(1).m = 1.377; % Massa do shoulder_link
133 cmass1 = T1.T*[-2.3E-05; -0.010364; -0.07336; 1];
134 L_alterado(1).r = cmass1(1:3); % Centro de massa

```

```

135 L_alterado(1).I = [0.00457, 0.004831, 0.001409, 1E-06, 0.000448, 2E-06]; % Inercia
136
137 L_alterado(2).m = 1.163; % Massa do half_arm_1_link
138 cmass2 = T2.T*[-4.4E-05; -0.09958; -0.013278; 1];
139 L_alterado(2).r = cmass2(1:3);
140 L_alterado(2).I = [0.011088, 0.001072, 0.011255, 5E-06, -0.000691, 0];
141
142 L_alterado(3).m = 1.163; % Massa do half_arm_2_link
143 cmass3 = T3.T*[-4.4E-05; -0.006641; -0.117892; 1];
144 L_alterado(3).r = cmass3(1:3);
145 L_alterado(3).I = [0.010932, 0.011127, 0.001043, 0, 0.000606, -7E-06];
146
147 L_alterado(4).m = 0.93; % Massa do forearm_link
148 cmass4 = T4.T*[-1.8E-05; -0.075478; -0.015006; 1];
149 L_alterado(4).r = cmass4(1:3);
150 L_alterado(4).I = [0.008147, 0.000631, 0.008316, -1E-06, -0.0005, 0];
151
152 L_alterado(5).m = 0.678; % Massa do spherical_wrist_1_link
153 cmass5 = T5.T*[1E-06; -0.009432; -0.063883; 1];
154 L_alterado(5).r = cmass5(1:3);
155 L_alterado(5).I = [0.001596, 0.001607, 0.000399, 0, 0.000256, 0];
156
157 L_alterado(6).m = 0.678; % Massa do spherical_wrist_2_link
158 cmass6 = T6.T*[1E-06; -0.045483; -0.00965; 1];
159 L_alterado(6).r = cmass6(1:3);
160 L_alterado(6).I = [0.001641, 0.00041, 0.001641, 0, -2E-06, 0];
161
162 % Inicializar a inercia do motor para todas as juntas como zero
163 for i = 1:7
164     L_alterado(i).Jm = 0;
165 end
166
167 % Adicao de uma massa adicional de 10kg no "end effector link"
168 L_alterado(7).m = 10.364;
169
170 % Calculo do centro de massa alterado
171 cmass_alterado = (0.364 * cmass7(1:3) + 10 * [0; 0; 0.1674]) / 10.364;
172 L_alterado(7).r = cmass_alterado;
173
174 % Inercia devido a massa adicional (em forma de vetor)
175 inercia_massa_adicional = (0.1674 - cmass_alterado(3))^2 * 10 * [1, 1, 0, 0, 0, 0];
176
177 % Inercia antiga no formato de matriz
178 inercia_antiga = [0.000214, 0.000223, 0.00024, 0, -2E-06, 1E-06];
179 I_A = [inercia_antiga(1), inercia_antiga(4), inercia_antiga(6);
180         inercia_antiga(4), inercia_antiga(2), inercia_antiga(5);

```

```
181         inercia_antiga(6), inercia_antiga(5), inercia_antiga(3)];
182
183 % Vetor de deslocamento entre os centros de massa antigo e alterado
184 d = cmass_alterado - cmass7(1:3);
185
186 % Matriz de inercia no novo ponto (teorema dos eixos paralelos)
187 I_P = I_A - 0.364 * ((d * d') - (d' * d) * eye(3) );
188
189 % Conversão da matriz de inercia alterada para o formato de vetor
190 inercia_massa_antiga_vec = [I_P(1,1), I_P(2,2), I_P(3,3), I_P(1,2), I_P(2,3), I_P(1,3)];
191
192 % Soma da nova inercia no formato de vetor
193 L_alterado(7).I = inercia_massa_adicional + inercia_massa_antiga_vec;
194
195 % Criando o modelo dinâmico do robo Kinova Gen3
196 kinova_alterado = SerialLink(L_alterado, 'name', 'Kinova Gen3 Alterado');
197
198 % Definir a transformacao base do robo
199 kinova_alterado.base = Tb0;
```